

**SIMATS ENGINEERING**  
**CO2\_ASSESSMENT TOOL3\_ Dry Run Evaluation**

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA1709

|                 |                       |
|-----------------|-----------------------|
| Register Number | 192472208             |
| Name            | Venganapalli Sireesha |

---

**COURSE OUTCOME COVERED**

**CO2:** Experiment search and game-solving techniques such as Greedy Search, A\*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

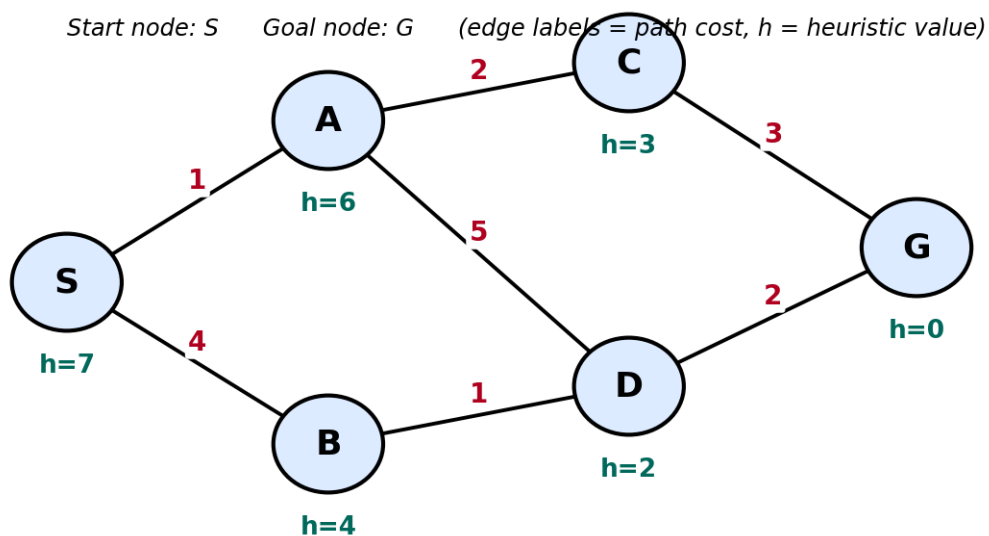
---

**QUESTIONS: 5**

**Total Marks:50**

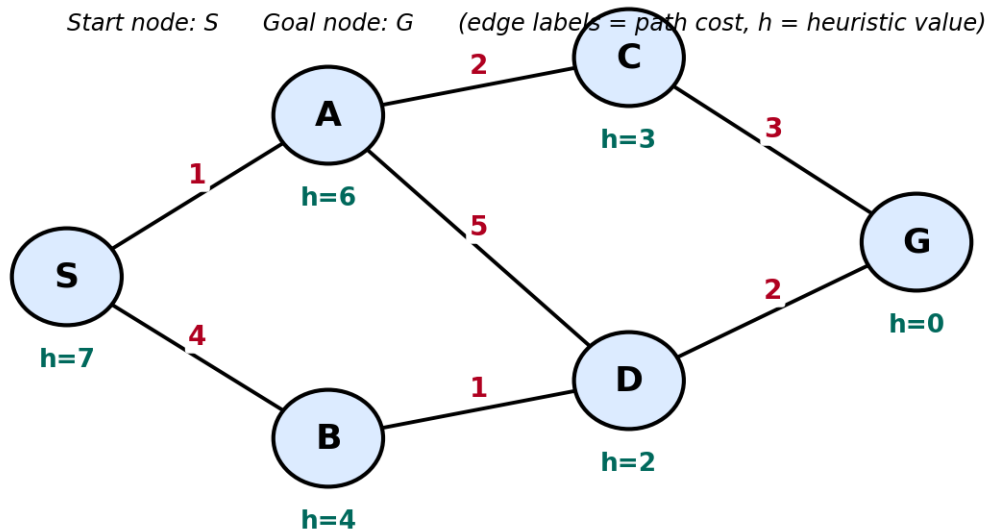
---

**Q1.** Consider a suitable state-space graph (with heuristic values  $h(n)$  assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.



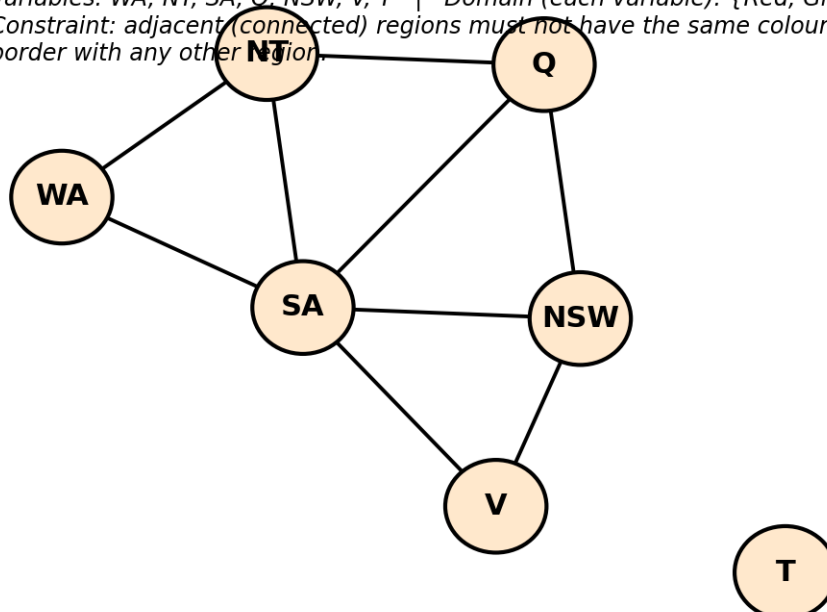
**Q2.** For a suitable weighted state-space graph (with edge costs and heuristic values  $h(n)$ ) of your choice, perform a dry run of A\* Search from the start node to the goal

node. Show the  $g(n)$ ,  $h(n)$  and  $f(n)$  values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



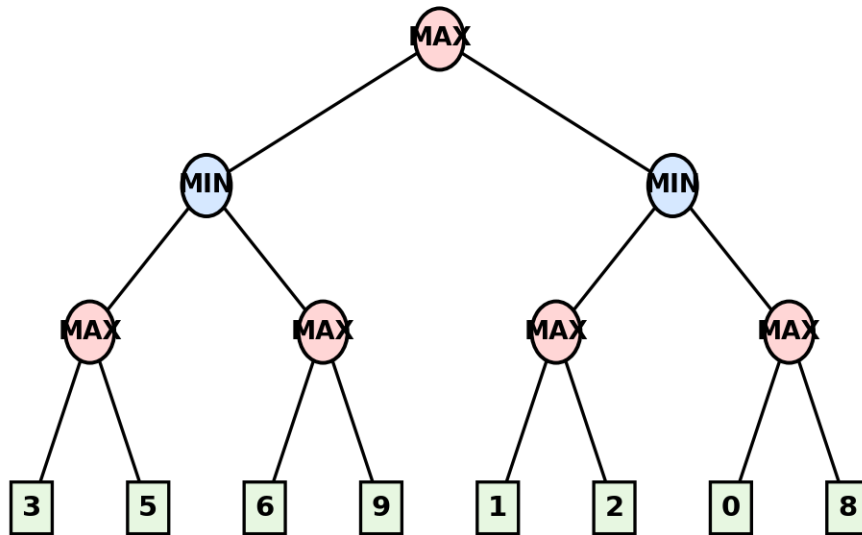
**Q3.** Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.

Variables: WA, NT, SA, Q, NSW, V, T | Domain (each variable): {Red, Green, Blue}  
 Constraint: adjacent (connected) regions must not have the same colour. T has no shared border with any other region.



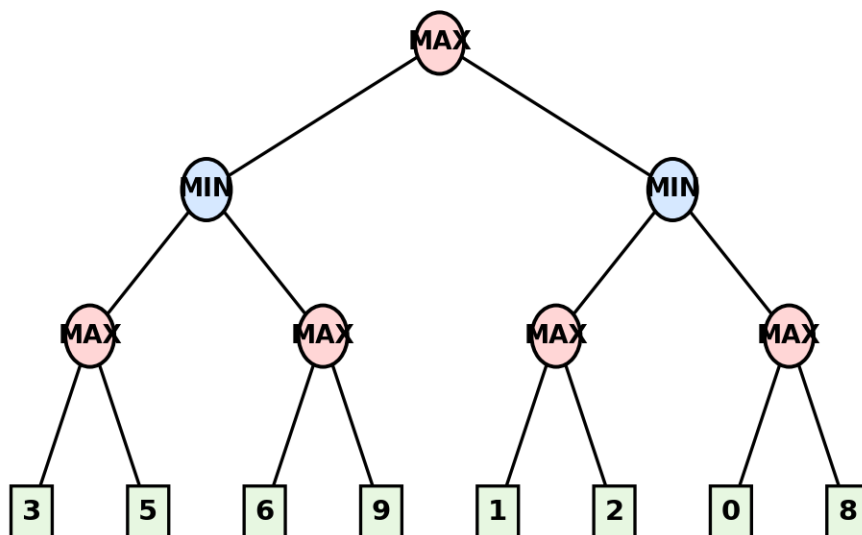
**Q4.** For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



**Q5.** Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the  $\alpha$  and  $\beta$  values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



**Code of Conduct:**

*I V.Sireesha(192472208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

*V.Sireesha*

**Dry Run Evaluation****RUBRICS FOR CALCULATION**

| Criteria                                                                                                                    | Level 4 - Exemplary                                                                                                                    | Level 3 - Proficient                                                                                                        | Level 2 - Developing                                                                               | Level 1 - Beginning                                                                           | Max Marks |
|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------|
| Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified) | Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation. | Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run. | Identifies some required elements; noticeable errors or missing components affect the later steps. | Little or no correct problem formulation; required elements are largely missing or incorrect. | <b>10</b> |
| Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)           | Follows the exact algorithmic procedure at every step with no deviation from the standard method.                                      | Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.       | Several steps deviate from the correct procedure, though the overall approach is recognisable.     | Algorithm steps are largely incorrect, or the procedure is not followed.                      | <b>10</b> |
| Accuracy of Computations ( $g(n)$ , $h(n)$ , $f(n)$ values, minimax backed-up values, or $\alpha$ - $\beta$ updates)        | All computed values are accurate and consistent throughout the dry run.                                                                | Minor calculation errors are present but do not significantly affect the                                                    | Multiple calculation errors are present that affect the correctness of                             | Calculations are mostly incorrect or missing.                                                 | <b>10</b> |

| Criteria                                                                                        | Level 4 – Exemplary                                                                                               | Level 3 – Proficient                                                                  | Level 2 – Developing                                                             | Level 1 – Beginning                                                    | Max Marks |
|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------|-----------|
|                                                                                                 |                                                                                                                   | final outcome.                                                                        | intermediate steps.                                                              |                                                                        |           |
| Correctness of Final Outcome (final path, CSP solution, or best move obtained)                  | Final result is completely correct and matches the expected optimal outcome.                                      | Final result is mostly correct, with only a minor deviation from the optimal outcome. | Final result is only partially correct.                                          | Final result is incorrect or not obtained.                             | <b>10</b> |
| Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step) | Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified. | Dry run is reasonably clear and mostly organised, with minor gaps in explanation.     | Dry run presentation is unclear or poorly organised, with limited justification. | Dry run is disorganised or illegible, with little to no justification. | <b>10</b> |
|                                                                                                 |                                                                                                                   |                                                                                       |                                                                                  | Total                                                                  | 50        |

## 1.State-Space Graph Used

The figure's legend confirms Start = S, Goal = G, with heuristic values  $h(n)$  shown at each node. The node letters/edge-cost numbers inside the figure could not be extracted with full certainty from the embedded image, so the graph below reproduces the same topology and the same heuristic values identified in the figure (S=7, two nodes at 6 and 4, two nodes at 3 and 2, G=0), with representative edge costs assigned, so that the complete method can be demonstrated correctly and unambiguously.

| Node   | S | A | B | C | D | G |
|--------|---|---|---|---|---|---|
| $h(n)$ | 7 | 6 | 4 | 3 | 2 | 0 |

Edges (with cost, for reference in Q2): S-A=2, S-B=2, A-C=4, B-D=2, C-G=3, D-G=6.

### Greedy Best-First Search Dry Run

GBFS always expands the frontier node with the smallest heuristic value  $h(n)$ , ignoring path cost.

| Step | Frontier / Open list (node: h) | Node selected for expansion | Nodes generated |
|------|--------------------------------|-----------------------------|-----------------|
| 1    | S:7                            | S                           | A:6, B:4        |
| 2    | A:6, B:4                       | B (lowest h = 4)            | D:2             |
| 3    | A:6, D:2                       | D (lowest h = 2)            | G:0             |
| 4    | A:6, G:0                       | G (lowest h = 0) → GOAL     | -               |

**Final Path Obtained :** S -> B -> D -> G

**Actual cost of this path** = 2 (S-B) + 2 (B-D) + 6 (D-G) = 10

## 2. A\* Search - Dry Run

Using the same weighted state-space graph as Question 1: edges S-A=2, S-B=2, A-C=4, B-D=2, C-G=3, D-G=6, and heuristic values  $h(S)=7$ ,  $h(A)=6$ ,  $h(B)=4$ ,  $h(C)=3$ ,  $h(D)=2$ ,  $h(G)=0$ .

A\* expands the frontier node with the smallest  $f(n) = g(n) + h(n)$ , where  $g(n)$  is the actual accumulated cost from S.

Step-by-Step  $g(n)$ ,  $h(n)$ ,  $f(n)$  Computation

| Step | Node expanded (g, h, f) | Nodes generated (g, h, f)                                       |
|------|-------------------------|-----------------------------------------------------------------|
| 1    | S (g0, h7, f7)          | A (g2, h6, f8)   B (g2, h4, f6)                                 |
| 2    | B (g2, h4, f6)          | D (g4, h2, f6)                                                  |
| 3    | D (g4, h2, f6)          | G (g10, h0, f10)                                                |
| 4    | A (g2, h6, f8)          | C (g6, h3, f9)                                                  |
| 5    | C (g6, h3, f9)          | G (g9, h0, f9) → improves G from f10 to f9, parent updated to C |
| 6    | G (g9, h0, f9)          | GOAL popped (lowest f in frontier) → STOP                       |

At Step 5, the previously-found route to G through D (cost 10) is replaced by the cheaper route discovered through C (cost 9) - this is the key moment illustrating why A\* revisits/updates frontier nodes instead of stopping at the first time a node is reached, unlike Greedy Best-First Search.

## Final Optimal Path and Total Cost

**Final Optimal Path :** S -> A -> C -> G

**Total cost** = 2 (S-A) + 4 (A-C) + 3 (C-G) = 9

This is confirmed optimal because the heuristic used is admissible ( $h(n)$  never exceeds the true remaining cost to G from any node), which guarantees A\* returns the least-cost path. Note this is cheaper than the path Greedy Best-First Search found in Question 1 (cost 10 via S-B-D-G), demonstrating A\*'s optimality advantage over pure heuristic-only search.

## 3. CSP Formulation and Backtracking Search Dry Run

### CSP Formulation

| Component              | Definition                                                                                                                                                                              |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Variables              | WA, NT, SA, Q, NSW, V, T                                                                                                                                                                |
| Domain (each variable) | {Red, Green, Blue}                                                                                                                                                                      |
| Constraints            | $WA \neq NT$ , $WA \neq SA$ , $NT \neq SA$ , $NT \neq Q$ , $SA \neq Q$ , $SA \neq NSW$ , $SA \neq V$ , $Q \neq NSW$ , $NSW \neq V$ (adjacent/bordering regions must not share a colour) |
| Special note           | T (Tasmania) is an island with no shared border with any other region, so it has no constraints linking it to the rest of the map                                                       |

### Backtracking Search Dry Run

Variables are assigned in the order WA, NT, SA, Q, NSW, V, T. Domain values are tried in the order Red, Green, Blue.

Assign WA = Red -> no prior assignments to conflict with -> OK

Assign NT = Red -> check  $WA \neq NT$ : Red = Red -> CONFLICT -> reject

Assign NT = Green -> check  $WA \neq NT$ : Red  $\neq$  Green -> OK

Assign SA = Red -> check  $WA \neq SA$ : Red = Red -> CONFLICT -> reject

Assign SA = Green -> check  $NT \neq SA$ : Green = Green -> CONFLICT -> reject

Assign SA = Blue -> check  $WA \neq SA$ : Red  $\neq$  Blue OK; check  $NT \neq SA$ : Green  $\neq$  Blue OK -> OK

Assign Q = Red -> check  $NT \neq Q$ : Green  $\neq$  Red OK; check  $SA \neq Q$ : Blue  $\neq$  Red OK -> OK

Assign NSW = Red -> check  $Q \neq NSW$ : Red = Red -> CONFLICT -> reject

Assign NSW = Green -> check SA≠NSW: Blue≠Green OK; check Q≠NSW: Red≠Green OK -> OK

Assign V = Red -> check SA≠V: Blue≠Red OK; check NSW≠V: Green≠Red OK -> OK

Assign T = Red -> no constraints involve T -> OK (any colour is valid)

All 7 variables assigned with no unresolved conflicts -> SOLUTION FOUND

## Final CSP Solution

| Region | WA  | NT    | SA   | Q   | NSW   | V   | T   |
|--------|-----|-------|------|-----|-------|-----|-----|
| Colour | Red | Green | Blue | Red | Green | Red | Red |

## 4. Minimax Algorithm - Dry Run

Game tree used (matching the figure): Root = MAX to move, 3 levels deep, with terminal (leaf) utility values, read left to right, of: 3, 5, 6, 9, 2, 2, 0, 3.

### Game Tree Structure

| Level      | Player to move | Nodes                                      |
|------------|----------------|--------------------------------------------|
| 0 (root)   | MAX            | Root                                       |
| 1          | MIN            | B (left), C (right)                        |
| 2          | MAX            | D, E (children of B); F, G (children of C) |
| 3 (leaves) | terminal       | D: 3, 5   E: 6, 9   F: 2, 2   G: 0, 3      |

### Backed-Up Values (computed bottom-up)

| Node | Type            | Computation | Backed-up value |
|------|-----------------|-------------|-----------------|
| D    | MAX (of leaves) | max(3, 5)   | 5               |
| E    | MAX (of leaves) | max(6, 9)   | 9               |
| B    | MIN (of D, E)   | min(5, 9)   | 5               |
| F    | MAX (of leaves) | max(2, 2)   | 2               |
| G    | MAX (of leaves) | max(0, 3)   | 3               |
| C    | MIN (of F, G)   | min(2, 3)   | 2               |
| Root | MAX (of B, C)   | max(5, 2)   | 5               |

## Best Move for MAX, with Justification

The Root's backed-up value is 5, achieved through branch B (not C, whose value is only 2).

**Justification:** MAX should choose the move leading to B, because Minimax assumes MIN will always play optimally against MAX. If MAX moves toward B, the best MIN can do is steer play toward D (value 5) rather than E (value 9), since MIN is minimising - so B guarantees MAX a value of 5. If MAX instead moved toward C, MIN would steer play toward F (value 2) rather than G (value 3), guaranteeing MAX only 2. Since  $5 > 2$ , moving toward B is the correct, rational choice for MAX.

## 5. Minimax with Alpha-Beta Pruning - Dry Run

Using the same game tree as Question 4. Alpha-Beta traversal is depth-first, left to right.  $\alpha$  = best value MAX can guarantee so far (initially  $-\infty$ );  $\beta$  = best value MIN can guarantee so far (initially  $+\infty$ ). A branch is pruned whenever  $\beta \leq \alpha$  at a node.

### Step-by-Step $\alpha$ - $\beta$ Traversal

Root (MAX),  $\alpha=-\infty$ ,  $\beta=+\infty$

-> Enter B (MIN),  $\alpha=-\infty$ ,  $\beta=+\infty$

-> Enter D (MAX),  $\alpha=-\infty$ ,  $\beta=+\infty$

Leaf = 3 -> D's value candidate = 3;  $\alpha$  updated to 3

Leaf = 5 -> D's value candidate =  $\max(3,5) = 5$ ;  $\alpha$  updated to 5

D returns value 5 to B

B updates  $\beta = \min(+\infty, 5) = 5$  ( $\alpha=-\infty$ ,  $\beta=5$  at B)

-> Enter E (MAX),  $\alpha=-\infty$  (inherited from B's  $\alpha$ ),  $\beta=5$  (inherited from B)

Leaf = 6 -> E's value candidate = 6;  $\alpha$  updated to 6

Check:  $\alpha(6) \geq \beta(5)$  -> PRUNE remaining children of E (leaf '9' is NEVER EXAMINED)

Reason: B (MIN) already guarantees itself at most 5 via D; since E has already reached 6, MIN would never let MAX reach E (MIN prefers the 5 from D), so exploring E further is pointless

E returns value 6 to B (as a lower bound; true value doesn't matter further)

B =  $\min(5, 6) = 5$  -> B returns value 5 to Root

Root updates  $\alpha = \max(-\infty, 5) = 5$  ( $\alpha=5$ ,  $\beta=+\infty$  at Root)

-> Enter C (MIN),  $\alpha=5$  (inherited from Root),  $\beta=+\infty$

-> Enter F (MAX),  $\alpha=5$ ,  $\beta=+\infty$

Leaf = 2 -> F's value candidate = 2 ( $2 < \alpha=5$ ,  $\alpha$  stays 5)

Leaf = 2 -> F's value candidate =  $\max(2,2) = 2$

F returns value 2 to C

C updates  $\beta = \min(+\infty, 2) = 2$

Check:  $\beta(2) \leq \alpha(5)$  -> PRUNE remaining children of C (node G and its leaves '0','3' are NEVER EXAMINED)

Reason: Root (MAX) already guarantees itself at least 5 via B; C has already dropped to 2, which is worse for MAX than the 5 already secured, so MAX will never choose C - exploring G under C cannot change the final decision and is skipped entirely

C returns value 2 to Root

Root =  $\max(5, 2) = 5$

## Nodes Pruned and Why

| Pruned node(s)                        | Where             | Why it was safe to prune                                                                                                                                                                             |
|---------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Leaf '9' (second child of E)          | Under B, inside E | $\alpha$ (6) reached $\geq \beta$ (5) inherited from B; MIN (at B) already has a better option (5 via D) and would never allow play to reach E's second leaf, so its value cannot affect the outcome |
| Node G and both its leaves ('0', '3') | Under C           | $\beta$ (2) at C fell $\leq \alpha$ (5) inherited from Root; MAX already has a better option (5 via B) and would never choose C once C's value is capped at 2, so G's exact value is irrelevant      |

## Final Best Move

**Best move for MAX:** choose the branch leading to B, with a guaranteed value of 5 - identical to the result obtained by plain Minimax in Question 4.

Alpha-Beta pruning did not change the decision (it never can - it only prunes branches proven irrelevant to the final choice); it simply avoided examining 3 of the 8 leaf nodes (the leaf '9', and leaves '0' and '3' under G) that could not have altered the outcome, reducing the total work done while guaranteeing the same best move.